

Discovery Executive Summary

Project: test-branch · Generated: 13/08/2026, 16:42:35

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

This is a small React 18 / TypeScript single-page application (login form with auth service) comprising 5 non-test source files and 184 total lines of code. The codebase has no backend layer — it is a frontend-only SPA that calls an external API. The architecture is generally clean for its current size: a dedicated `authService` abstracts API calls, typed DTOs exist in `src/types/auth.ts`, and routing is handled via `react-router-dom`. Two moderate issues were identified: (1) the `LoginPage` component imports `axios` directly to use `axios.isAxiosError()` for error classification, creating a leaky abstraction that bypasses the service layer; and (2) `localStorage` access for token persistence is hard-coded inside the component rather than delegated to a service or hook, mixing concerns. No backend controllers, repositories, models, or database access exist in this codebase — backend-oriented hotspots (H1–H3, H6, H8–H9) are not applicable. **Layers covered:** Frontend (React/TypeScript, 5 source files). Backend: none detected.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	N/A (no backend controllers)	Good
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	N/A (no backend controllers)	Good
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 (no database access)	Good
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	0	Good
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	N/A (no database)	Good
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	0	Good
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	N/A (no database)	Good
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	130 LOC avg (LoginPage 130, Dashboard ~4)	Good
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	1 component (LoginPage imports axios directly + direct localStorage)	Moderate
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	Good
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	0 levels (no prop drilling, no global state)	Good

F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	0 (all functional components with hooks)	Good
----	--	---------------------------	---	------	-----	--	---------------------------------------

1.4 Actions Required

Hotspot	Action	Rating	Priority
F2 — Missing Frontend Service/Data Layer	Move error classification (<code>axios.isAxiosError</code> + status code checks) into <code>authService</code> as typed application errors; extract <code>localStorage</code> token operations into a <code>tokenStorage</code> utility or <code>authService</code> methods (<code>storeToken</code> / <code>getToken</code> / <code>clearToken</code>); remove direct <code>axios</code> import from <code>LoginPage.tsx</code>	Moderate	Medium

1.5 Expected Outcomes

- Clean abstraction boundary:** LoginPage will depend only on `authService` and `tokenStorage` interfaces, not on `axios` internals — enabling HTTP client swaps without touching view code.
 - Reusable token management:** A `tokenStorage` utility enables consistent token handling across future pages (logout, token refresh, auth guards) without duplicating localStorage logic.
 - Improved testability:** Components can be tested without mocking `axios` or `localStorage` directly — mock the service/utility instead, which is already the pattern used in `authService.test.ts`.
 - Foundation for growth:** As the app grows beyond the login page, the service-layer pattern is already established — new features can follow the same `component → service → typed DT0` architecture without accumulating leaky abstractions.
- ```

{
 "stop_reason": "end_turn",
 "session_id": "4f90cafb-d0fc-4bed-ada8-d467531bbcc2",
 "total_cost_usd": 1.2760355000000003,
 "usage": {
 "input_tokens": 14,
 "cache_creation_input_tokens": 64707,
 "cache_read_input_tokens": 570099,
 "output_tokens": 13330,
 "server_tool_use": {
 "web_search_requests": 0,
 "web_fetch_requests": 0,
 "service_tier": "standard",
 "cache_creation": {
 "ephemeral_1h_input_tokens": 64707,
 "ephemeral_5m_input_tokens": 0,
 "inference_geo": "not_available",
 "iterations": [
 {
 "input_tokens": 1,
 "output_tokens": 1576,
 "cache_read_input_tokens": 60203,
 "cache_creation_input_tokens": 4504,
 "cache_creation": {
 "ephemeral_5m_input_tokens": 0,
 "ephemeral_1h_input_tokens": 4504,
 "type": "message"
 }
 }
],
 "speed": "standard",
 "modelUsage": {
 "claude-haiku-4-5-20251001": {
 "inputTokens": 10516,
 "outputTokens": 16,
 "cacheReadInputTokens": 0,
 "cacheCreationInputTokens": 0,
 "webSearchRequests": 0,
 "costUSD": 0.010596,
 "contextWindow": 200000,
 "maxOutputTokens": 32000,
 "opus-4-6": {
 "inputTokens": 14,
 "outputTokens": 13330,
 "cacheReadInputTokens": 570099,
 "cacheCreationInputTokens": 64707,
 "webSearchRequests": 0,
 "costUSD": 1.2654395000000003,
 "contextWindow": 200000,
 "n": 1,
 "terminal_reason": "completed",
 "fast_mode_state": "off",
 "uuid": "a6109447-cf3b-462a-b1ef-d861190bfc5"
 }
 }
 }
 }
 }
 }
}
```